

Implementačná úloha 1. (40 bodov)

Implementujte RESTful servis, ktorý bude sprístupňovať informácie poistných zmlúv a poistených osobách. Informácie o poistnej zmluve predstavujú resource, ktorého stav reprezentovaný vo formáte XML má štruktúru:

```
<?xml version="1.0" encoding="UTF-8"?>
<poistenie>
  <idZmluvy>Z123</idZmluvy>
  <majitel> Jozef Mrkvicka </majitel>
  <poistenaOsoba>Ferdo Mrkvicka< poistenaOsoba >
  <poistenaOsoba >Janko Hrasko</poistenaOsoba>
</poistenie>
```

kde:

- element **idZmluvy** obsahuje jednoznačný identifikátor zmluvy. Teda musí byť zadany.
- element **majitel** obsahuje meno majiteľa zmluvy.
- elementy **poistenaOsoba** obsahujú mená poistených osôb.

Pozn. Uvedený dokument je ukážka. XML dokumenty, s ktorými servis pracuje budú mať elementy s rovnakými názvami a štruktúrou, odlišovať sa budú len textovým obsahom a počtom elementov. (poistených osôb môže byť viac ale nemusí byť žiadna. Poradie elementov nie je dôležité.)

POZOR dôležité!

- Vytvorte **java web application** project. (môžete pracovať aj s maven projektom, nepoužívajte však žiadne špeciálne dependencie okrem implementácie rest-api napr. jersey)
- Package pre resource nazvite: **rest** a vytvárajte v ňom aj všetky ostatné (dátové) triedy.
- Path – koreňový resource nazvite: **poistenie**

Pozn. Pre anotáciu @Singleton importujte **javax.inject.Singleton**

Nezabudnite na anotáciu @XmlElement

Servis inicializujte tak, že po štarte (deploymente) bude vytvorený jediný resource, a to pre poistenie s id **Z123**, majiteľom “**Jozko Mrkvicka**”, **bez poistených osôb**. Ressourcy pre ďalšie poistné zmluvy bude možné pridávať pomocou metódy POST.

1. Pre koreňový resource **poistenie** implementujete metódu

POST, ktorá slúži pre vytvorenie nového resoursu s informáciami o poistnej zmluve.

Metóda akceptuje XML dokument (MIME: APPLICATION_XML) podľa horeuvedenej špecifikácie.

Preverí, či zmluva z daným identifikátorom už neexistuje. Ak neexistuje, vytvorí nový resource, inak neurobí nič. Metóda nič nevracia.

2. Všetky informácie o poistnej zmluve sú prístupné ako subresource s URI **poistenie/{id}**, kde reťazec **id** je identifikátor zmluvy.

Pre tento resource implementujte nasledujúce metódy:

GET pre MIME TEXT_PLAIN: vráti počet poistených osôb. Ak resource neexistuje, malo by sa vrátiť **0** prípadne HTTP 204 alebo HTTP 404 .

GET pre MIME APPLICATION_XML: vyhľadá zmluvu podľa identifikátora zadaného v URL a vráti informácie ako XML dokument s horeuvedenou štruktúrou. Ak zmluva neexistuje, malo by sa vrátiť HTTP 204 alebo 404 (Pomôcka, metóda vráti jednoducho null)

POST slúži na pridanie ďalšej poistenej osoby do zmluvy. Očakáva reťazec s menom osoby (MIME: TEXT_PLAIN) a vracia taktisto textový reťazec (MIME: TEXT_PLAIN).

Ak resource pre zmluvu neexistuje, vráti metóda text “**neznama zmluva**”

Ak je osoba s daným menom ešte nie je medzi poistenými osobami v zmluve, pridá ju medzi poistené osoby a vráti reťazec “**OK**”

Ak osoba už bola poistená vráti takisto “**OK**” ale ju nepridáva znovu do zoznamu.

3. Servis umožňuje tiež vyhľadať všetky zmluvy, na ktorých je osoba so zadaným menom poistená. Túto funkcionality implementujte pomocou metódy **GET** pre koreňový resource **poistenie**. Meno osoby je zadané ako parameter požiadavky **osoba** (*@QueryParam("osoba") - pozri ťahák*). Metóda nájde všetky zmluvy, v ktorých je osoba medzi poistenými a vráti zoznam identifikátorov zmlúv, oddelených medzerou. (MIME: TEXT_PLAIN)

Ak meno osoby nie je zadané ako parameter požiadavky, nevráti nič, príp. prázdny reťazec.

Pokyny pre odovzdanie:

Do miesta odovzdania **t2-u1rest** sa odovzdávajú **všetky vytvorené zdrojové súbory**, t.j. všetky súbory z adresára **rest.**, samostatne, t.j. **NEZOZIPOVANÉ!**

Implementačná úloha 2. (26 bodov)

Implementujte SAX-aplikáciu na parsovanie XML dokumentov s informáciami o štúdiu (študijných zameraniach, prednáškach a vyučujúcich). Aplikácia pomocou SAX-handlera dokument prečíta a **vypíše na obrazovku názvy prednášok v zimom semestri, ktoré učí vyučujúci so zadaným menom**.

Maven projekt **saxapp** pre vývoj aplikácie máte už vytvorený a pribalený v zip-súbore so zadáním. Projekt obsahuje 2 zdrojové súbory a jeden vzorový XML súbor:

- **main/java/saxapp/SaxHandler.java**: Nedokončená implementácia handlera. Obsahuje len
 - konštruktor s jedným parametrom slúžiacim na zadanie mena vyučujúceho,
 - neimplementované metódy handlera.
- **main/java/saxapp/SaxApp.java**: Hlavný program aplikácie slúžiaci na otestovanie handlera.
- **test/data/studium.xml**: Ukážka vzorového xml-dokumentu, s informáciami o štúdiu.

Projekt otvorte vo vývojovom prostredí a **dokončíte implementáciu handlera** (SaxHandler.java) podľa horeuvedenej špecifikácie.

Pokyny k implementácii.

- Program implementujte bez použitia dynamických dátových štruktúr (ako list, map,... či iných java-kontajnerov)
- V súbore **SaxHandler.java** stačí doimplementovať metódy handlera.
(Samozrejme môžete pridať aj deklarácie ďalších privátnych dátových členov. Nič iné, nie je potrebné odstraňovať ani modifikovať).
Tento súbor treba odovzdať.
- Súbor **SaxApp.java** je hotový program, ktorý načíta súbor **studium.xml** a spracuje ho pomocou SaxHandlera. Je určený len pre vaše testovanie, ak chcete, môžete si ho upraviť.
Neodovzdáva sa!
- Súbor **studium.xml** je len ukážkou xml-dokumentu, s informáciami o štúdiu.
Handler musí vedieť správne spracovať aj dokumenty, ktorých textový obsah elementov je iný.
Mal by počítať tiež s tým, že podelementy **vyucujuci**, **prednaska** ani atribút **semester** nie sú povinné (môžu chýbať) a poradie elementov môže byť iné ako je v ukážke.
Pre dôkladnejšie testovanie môžete súbor **studium.xml** modifikovať, prípadne vytvoriť ďalšie, podobné xml súbory.

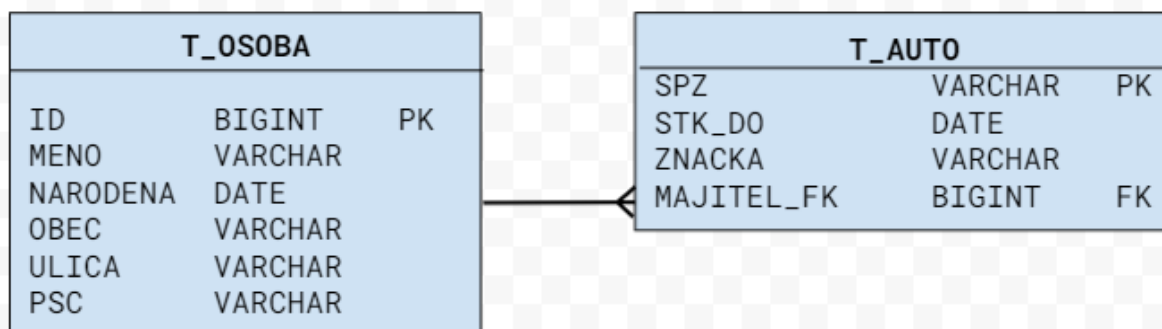
Odovzdáva sa len zdrojový súbor **SaxHandler.java** do miesta odovzdania **t2-u2sax**.

Úloha 3. (14 bodov)

Zip-súbor so zadáním obsahuje adresár **entities** s tromi súbormi:

- **Adresa.java**: Dátová trieda reprezentujúca informácie o adrese.
- **Osoba.java**: Dátová trieda reprezentujúca informácie o osobe.
- **Auto.java**: Dátová trieda reprezentujúca informácie o aute.

Vašou úlohou je **doplniť** do týchto tried **JPA-annotácie** tak, aby databáza, na ktorú sa triedy mapujú, (t.j. vygenerovaná na základe anotácií) mala **presne takú štruktúru**, ako zobrazuje **diagram**:



Existujúci kód v súboroch nie je potrebné modifikovať, **názvy a typy dátových členov nemeniť!**

Stačí ho **doplniť len o potrebné JPA anotácie**, tak aby boli triedy namapované na databázové tabuľky so štruktúrou, ako je uvedená v diagrame (zhodovať sa musia predovšetkým mená tabuliek i stĺpcov) a pritom:

- ID - kľúč tabuľky OSOBA bude **autogenerovaný**
- SPZ - kľúč tabuľky AUTO nie je autogenerovaný.

Pozn. importy pre anotácie môžete tiež doplniť, nie je to však nutné (nebudú sa kontrolovať).

Pokyny pre odovzdanie:

Do miesta odovzdania **t2-u3db** sa odovzdávajú sa zdrojové súbory:

- **Osoba.java**
- **Auto.java**
- **Adresa.java**

samostatne, t.j. **NEZOZIPOVANÉ!**